# Report of YASM: Yet Another Similarity Measure Method

Yan Meng
8/12/2002

## Abstract

Content based information retrieval has evolved for a long time; yet, how to precisely measure the perceptual similarity between two examples is still an open issue. Some distance functions based on MinKowski metric have been studied to measure people's perceptual characteristics. However, based on our extensive experimental results, MinKowski metric does not work effectively in finding obvious similar examples. In this study, we propose a new distance function—YASM (Yet Another Similarity Measure Method) to measure perceptual similarity, and psychological explanation of YASM function has also been studied. After comparing with another advanced method—DPF (Dynamic Partial Distance Function) in image retrieval, we conclude that YASM works better both in effectiveness and efficiency.

Keywords: similarity measure, perceptual distance function, image retrieval

## 1 Introduction

With advances of the technology and the emergency of large scale multimedia applications, content-based image retrieval has become one of the most challenging and important research directions in the recent decades. To design such a system, one needs to address at least two issues: the first is to find effective and compact representations for the images; the second is to select distance metrics for data ranking according to the human perception of the contents. The latter is the focus of this paper. For this purpose, we will assume the selected feature representations are effective to capture visual perception. Many distances, such as Minkowski metric [1], Earth mover distance [2], Fuzzy Logic [3], and DPF [4], have been used to measure similarity between image feature vectors. However, our experiments show that they frequently failed to find some obviously similar images and are not adequate in measuring perceptual similarity. Hence, it is still an open question on how to measure the perceptual similarity.

Similarity measure plays a central role in theories of information retrieval. In this paper, we discovered a new distance function to measure the similarity of human perception. We call it Yet Another Similarity Measure Method (YASM). This new method itself is non-metric, which is consistent with theories of Tversky[5] who shows that perceptual distances can be non-metric. In our method, only the similar feature channels are

selected as the interesting features to measure the similarity, while the dissimilar ones are discarded as noise. This is consistent with human's perception in the process of similarity measure [6, 9]—Similarity is the process that determines the respects for measuring similarity. To closely simulate human perception in selecting similar respects, we use the concept of Just Noticeable Difference (JND) from psychological theories [6], and the term of Just not the Same (JNS) [7] to determine the possibility of the similarity. If the difference of two objects in a channel is within JND, we regard them as the same in this respect. If the difference is between JND and JNS, we treat them as similar in this respect. If the difference is greater than JNS, we consider them as dissimilar in this respect. When we empirically compare YASM with DPF and Minkowski-type distance functions in image retrieval application, YASM works much better than either of them.

The rest of the paper is organized in the following way: section 2 describes the discovery of YASM. And Section 3 depicts the construction of the testbed and feature selection. Experimental results in Section 4 demonstrate that YASM outperforms DPF and Minkowski-type metrics in measuring image similarity. Finally, we offer our conclusion and discuss some future research directions in Section 5.

# 2 YASM

## 2.1 Traditional methods and limitations

### 2.1.1 Minkowski distances

The Minkowski metric (Equation 1) and its weighted version (Equation 2) are widely used for measuring image similarity. When r is 2, 1, and $\infty$, it defines L2 (Eculidean Distance), L1, and $L_\infty$ norm respectively. Mathematically, it is convenient to compute the distances between objects. However, such metrics assume that the similar objects should be similar in the same set of features, which is not appropriate in measuring similarity in human perception space[7, 8].

$$D(q,p) = \left(\sum_{i=1}^{k} \left|q_i - p_i\right|^r\right)^{\frac{1}{r}}, \qquad \text{(Equation 1)}$$

$$D(q,p) = \left(\sum_{i=1}^{k} w_i \left|q_i - p_i\right|^r\right)^{\frac{1}{r}} \qquad \text{(Equation 2)}$$

### 2.1.2 More alternatives

Tversky shows that perceptual distances can be non-metric. Based on this idea, [4] recently studied a Dynamical Partial distance Function (DPF) to measure image similarity (Equation 4). It introduced the ideas that image similar to the query images can be similar in differing features, and that similar images do not resemble the query images in all features. All the feature distances are first sorted and a fixed number of smallest feature distances are chosen to measure the similarity. But this fixed number of feature distances may actually involve some dissimilar channels (noise) or ignore some similar

channels in the process of human perception. For example, after training a large dataset, we find the best m value is 114 for the whole dataset, but for each image, it may not be the best value. If the number of similar channels for image A to the query is 100, using 114 as the best value may involve 14 dissimilar channels, which are noisy. On the contrary, if the number of similar channels for image B to the query is 130, using 114 as the best value may ignore 16 similar channels, which also contribute to the similarity.

$$D(q,p) = \Big(\sum_{|q_i - p_i| \in \Delta_m} |q_i - p_i|^r\Big)^{\frac{1}{r}}, \qquad \text{(Equation 4)}$$

## 2.2 YASM

Instead of confining ourselves to the traditional settings of the distance functions, a better way is to use a new form of the distance function to suit our objective, for which we believe that the similarity measure of human perception is better cast as "a process of seeking out the respects for measuring similarity".

### 2.2.1 Psychological interpretations of YASM

From psychological view, we know that attention is the gateway to the information processing, and "people's attention is the process of seeking out stimuli that are of interest to them"[6, 9]. This indicates that certain features are selected to be processed, and other features are ignored in a very real sense. In similarity measure, people's interest is the objects' similarity, and they seek out similar features as the interesting respects. This will result in that for different object pairs, they may have different similar channels as well as different number of similar channels. For example, when people go to a car market, they select only the interesting features of the cars, and ignored other uninteresting features (noise). People may say that the gas-saving quality and the long endurance are their interests for Nissan, and the powerful engine, the well-known brand and comfortable seat for BMW. The variation in similar channels and number of channels for different object pairs motivates our discovery of YASM.

So, the next question is how does people select similar channels? To answer this question, we use two terms to capture people's ability to seek out similar channels. One is the just noticeable difference (JND), which defines the smallest difference between two stimuli that a person can detect [6]. We also introduce another term -- just not the same (JNS), which defines a threshold-- when the objects' difference is larger than the threshold, they will be considered as different. For example, suppose for the objects, we only compare the weight channel and the other channels such as color and size are regarded as the same. A person can tell the weight difference between 100-gram and 105-gram, but we cannot detect a smaller distance. So the JND of 100-gram for this person is 5 grams. And, a person may say that 100-gram is just not the same as a weight more than 120-gram. So the JNS of 100-gram for this person is 20gram. In YASM, based on above analysis of the similarity measure of the human perception, feature distances are categorized into three groups:

When a feature distance of two objects in channel A is beyond JNS, the similarity of two objects in this channel is defined zero — totally different.

When a feature distance of two objects in channel A is between JND and JNS, the similarity of two objects in this channel is between zero and one -- similar.

When a feature distance of two objects in channel A is below JND, the similarity of two objects in this channel is defined one – almost the same.

In this way to measure the similarity, for different object pairs, people may find different similar channels and different number of similar channels, which makes our similarity measure difficult. Then how could we deal with this problem? Similar to traditional methods, we first normalize each feature channel using Gaussian normalization, which ensures the equal emphasis of the feature components within a feature vector. For instance, the feature distance in channel A is 0.2, it is considered the same as the feature distance 0.2 in channel B. This means that after normalization, the contribution of each channel to the similarity measure is the same. (Later, we also use the weighted method as what MARS did [10].). Then we average all the similar channels, and use the mean value as the contribution of all similar channels to the similarity measure. Another reason of using averaging to approximate the optimal distance is that in this way we could also take into consideration some hidden similar features, which can not be extracted using our feature extractor. (Since each object may include millions of features, and we only extract some of them, there are still lots of hidden features that could contribute to the similarity.) To further consider that the more similar channels two objects have in common, the more similar they are, we also need to take into account the number of similar channels in our distance function.

### 2.2.2 YASM distance function

Suppose each object is represented by a $k$ dimensional feature vector $\{x_1, x_2, ..., x_k\}$. The distance function to represent the perceptual similarity of two objects $p$ and $q$ is defined as (Equation 5):

$$D(p,q) = \left(\frac{1}{\left(N_{|p_j - q_j| \le JND} + N_{JND < |p_i - q_i| \le JNS}\right)^l} \sum_{i \in A_{JNS}} |p_i - q_i|^r\right)^{\frac{1}{r}}, \qquad 1 \le i \le k, l > 1, \quad \text{(Equation 5)}$$

where $N_{JND}$ is the number of feature distances $|p_i - q_i|$ that are less than JND, and $N_{JNS}$ is the number of feature distances that are within the range of JND and JNS, and $A_{JNS}$ is the feature set within which the feature distance $|p_i - q_i|$ is between JND and JNS, and the following condition (Equation 6) must be satisfied.

$$\begin{cases} N_{|p_i - q_i| \le JND} \le k \\ N_{JND < |p_i - q_i| \le JNS} \le k \\ N_{|p_i - q_i| \le JND} + N_{JND < |p_i - q_i| \le JNS} \le k \end{cases}, \qquad \text{(Equation 6)}$$

In similarity measure, we only consider two types of channels: those channels with feature distances less than JND, which are treated as the same ($|p_i - q_i| = 0$), and those channels with feature distances between JND and JNS. Channels with feature distances larger than JNS are discarded as dissimilar channels.

Instead of using averaging, we use the $(N_{|p_j - q_j| \le JND} + N_{JND < |p_i - q_i| \le JNS})^l, l > 1$ as the denominator since in some scenario, averaging cannot work well. For example, suppose there are three similar channels between A and B, and the difference for each channel is 0.1, using average, we get the distance of A and B is (0.1+0.1+0.1)/3 = 0.1; while in another case, suppose the number of similar channels between A and C is 5, and the difference for each channel is also 0.1, then the distance of A and C is (0.1+0.1+0.1+0.1+0.1)/5 = 0.1. But under our assumption that the more similar channels two objects have in common, the more similar they are. We think A is more similar to C than B. Using $(N_{|p_j - q_j| \le JND} + N_{JND < |p_i - q_i| \le JNS})^l, l > 1$, we could get the distance of A and B is (0.1+0.1+0.1)/9 = 0.033, and the distance of A and C is (0.1+0.1+0.1+0.1+0.1)/25 = 0.02. After emphasizing the number of similar channels, our distance functions can get even closer to the real cases. Later in our experiments, we use $l = 2$.

### 2.2.3 Parameters: JNS and JND

There are two parameters in this function: JNS and JND. The effect of JNS to similarity measure can be explained in this way: very small JNS means that people's similarity measure is very restrictive, and only those almost the same images could be retrieved; very large JNS means the similarity measure is loose, and some images not very similar will be retrieved. Also, considering the diversity of users' JNS, we need to choose an optimal JNS value for each user. For JND, when it is very small, means a person is very sensitive to the small differences. This person is able to find the almost identical objects and easily discard less similar ones. When JND becomes larger, it implies that a person would include in his/her selection not only the almost identical objects but also the similar ones. But too large JND is not meaningful, under which a person consider almost all the objects as the same. Later these effects are further confirmed by our experimental results. From the above analysis, we can reach this understanding that there exist the optimal values of JNS and JND that can separate similar and dissimilar images efficiently. So in the following section, we will provide a method to search for JNS and JND.

### 2.2.4 Automatically adjusting JNS

Given a large image dataset, the JND is usually small after we normalize the data within the range of [0, 1]. Therefore we could assign it a small positive value. In our later experiments, we fixed it to be 0.02 based on our preliminary results.

To adjust JNS, since we know there is only one optimal JNS that could achieve highest precision under the same recall rate, we used greedy algorithm to find the optimal value.

Specifically, we employed an evaluation function, in which, three values, such as precision[recall=60%], precision[recall=70%], and precision[recall=80%] are evaluated given a JNS value. If at least two of them can achieve highest values at the same time, we will take this JNS value as the best setting. The algorithm works in the following way:

1. Initialize(precision[recall=60%], precision[recall=70%], precision[recall=80%]);
2. window_size = 0;
3. JNS ← 0.5; // we think those channels with feature distances larger than 0.5 as dissimilar channels
4. for (recall = 60%; recall <= 80%; recall += 10%)
   if (new_precision(recall, JNS) >= old_precision(recall, JNS_old))
   window_size ++;
5. if (window_size < 2)
6. then return JNS_old as optimal value;
7. else {
   a. updata JNS; // JNS ← JNS_old – JNS_STEP;
   b. Go to step 3
   }

Using this greedy algorithm, we could find the best JNS setting automatically. Later, we will consider using relevance feedback technique to learn users' personal concepts of JND and JNS.

## 2.3 Weighted YASM

The idea of YASM can also be generalized to weighted YASM (WYASM) to further improve the weighted Minkowski and weighted DPF. The WYASM is defined as (Equation 7):

$$D(p,q) = \left(\frac{w_i}{\left(N_{|p_j - q_j| \le JND} + N_{JND < |p_i - q_i| \le JNS}\right)^l} \sum_{i \in A_{JNS}} |p_i - q_i|^r\right)^{\frac{1}{r}}, \qquad 1 \le i \le k, l > 1, \text{ (Equation 7)}$$

# 3 Testbed construction and feature selection

In this section, we describe how our image dataset was constructed in two categories:
1. Testbed construction
2. Feature selection

## 3.1 Testbed Construction

The images come from 1,695 images collected from Corel image CDs and the Internet. These images cover 18 categories – *Architecture, Bear, Bugs, City, Couds, Elephanes, Fabrics, Fireworks, Flowers, Food, Landscape, People, Space, Textures, Tigers, Tools, Vehicles and Waves*.

For each image, we use it as the seed and conduct the following 50 transformations and obtain 1,695 similar image sets. The total number of images in the general dataset is 86,445.

The 50 transformations include:

1. Colorizing.
   We colorize each image through Red/Green/Blue channels by five percentages: 0/0/30%, 0/30%/0, 30%/0/0, 30/70%/90% and 50%/30%/0.
2. Modulation.
   For each image, we vary the amplitude of the hue/saturation/brightness channels by fifteen percentages: 40%/0/0, 60%/0/0, 80%/0/0, 120%/0/0, 140%/0/0, 0/40%/0, 0/60%/0, 0/80%/0, 0/120%/0, 0/140%/0, 0/0/40%, 0/0/60%, 0/0/80%, 0/0/120% and 0/0/140%.
3. Cropping.
   We symmetrically remove the outer borders to reduce each image by 5%, 10%, and 20%, and then scale it back up to the original size.
4. Rotation.
   We rotate each image by 90, 180 and 270 degrees.
5. Scaling.
   a. Scale up then down. We scale each image up by 5 and 10 times, and then scale it back to the original size.
   b. Scale down then up. We scale each image down by factors of 2, 5 and 10, then scale it back to the original size.
6. Downsampling.
   We downsample each image by seven different percentages: 10%, 20%, 30%, 40%, 50%, 70% and 90%.
7. Shading.
   We shade the image using a distant light source, the position of which is specified by the two parameters, azimuth =1000, elevation = 50.
8. Blurring.
   We blur each image using five Gaussian operators, of which the radius/standard deviations based on pixels are: 6/2, 6/3, 6/4, 12/4 and 12/8.
9. Changing format.
   We obtain the GIF version of each JPEG image.
10. Changing contrast.
    We enhance the intensity differences between the lighter and darker elements of each image.
11. Despeckling
    We despeckle each image.
12. Flopping
    We obtain the mirror image of each image.
13. Enhancing
    We enhance the intensity differences between the lighter and darker elements of each image.

## 3.2 Feature Selection

In our experiments, each image is represented by a 144-dimensional feature vector composed of three types of features: color, texture and shape [11, 12, 13, 14]. Colors are divided into 12 bins, *black, white, red, green, blue, brown, yellow, pink, purple, gray, orange* and an *outlier* bin. Each color bin is characterized with nine features: *color histogram, color means* (in *H*, *S* and *V* channels)*, color variance* (in *H*, *S* and *V* channels) and two shape characteristics: *elongation* and *spreadness*. Color elongation characterizes the shape of a color, and color spreadness characterizes how that color scatters within the image.

Texture is another important characteristic for image analysis. As shown by many researchers, characterizing texture features in terms of structuredness, orientation, and scale (coarseness) fits well with models of human perception. In our experiments, texture features are characterized in vertical, horizontal and diagonal orientations. In each orientation, there are 4 types of features: energy mean, variance, elongation and spreadness, each in coarse, medium and fine levels.

# 4 Empirical study

Our empirical study of the image retrieval includes five parts:

1. To demonstrate why YASM works better than traditional distance functions
2. To study the effect of the JND and JNS
3. To compare the performance of WYASM with YASM
4. To study the effect of the fractional distance functions
5. To study the robustness of YASM
6. Further analysis

## 4.1 Comparison of YASM with DPF and Minkowski-type distance functions

To compare DPF with YASM, we use the whole image dataset as the training data, and find the best M value of DPF -- 114, which is the same as [4] indicates. Then we use the training dataset as the testing data to compare DPF with YASM. In Fig 1, Fig 2, Fig 3, and Fig 4, we compared YASM with DPF, and Minkowski-type distance functions using different kinds of images as queries. From Fig 1, we can see that using seeding images as queries, in order to achieve the recall rate of 90%, the retrieval precision of L2 distance is about 1.3%, L1 distance about 1.8%, and DPF is better, about 7.3%, but YASM is the highest, about 12.5%. And using any kind of images as queries, YASM works the best.

To further explain why YASM works better than DPF, we study the feature distance distributions of similar and dissimilar images. We first tallied the feature distances of both similar and dissimilar images. Since we normalized the data to be in the range of [0, 1], the distances are also in the range of [0, 1]. Fig 5 shows that the distribution of

similar images is much skewed toward small values, while the dissimilar images are quite evenly distributed. This can explain why DPF works well after choosing top-M smallest feature distances to measure the similarity. But DPF only considers the general distributions of similar and dissimilar images in all the channels. Fig 6, and Fig 7 show that for different channels (we randomly chose two channels in the demonstration ), not only the distribution of similar images is still much skewed toward small values, but also the distribution of numbers of the small values are much different. For example, in the 101st channel, the numbers of feature distances between [0, 0.01) and [0.01,0.02) are 87 and 33 respectively. But in the 109th channel, they are 79 and 122 respectively. The 114th smallest value in the 101st channel is 0.0172, and the 130th smallest value in the 109th channel is 0.0143. So we can see that the 130th smallest value in 109th channel actually contributes more to the similarity measure then the 114th smallest value in 101st channel does (because after normalization, each channel is considered equally important). Since YASM also takes this phenomenon into account, it works even better. In Fig 6, we could see that the feature distances of the dissimilar images are randomly distributed in the 101st channel. This is because after we normalized the feature distances to be between zero and one, the dissimilar images are in Gaussian distribution. So, we conclude that DPF can only get a general good trade-off between similar and dissimilar images, and it may actually introduce some dissimilar channels in similarity measure, which will degrade its performance. But in YASM, all the similar channels are considered and all the dissimilar channels are discarded in measuring similarity. Based on above analysis, we summarize that YASM works better than other distance functions, such as DPF and Minkowski-type distance functions.

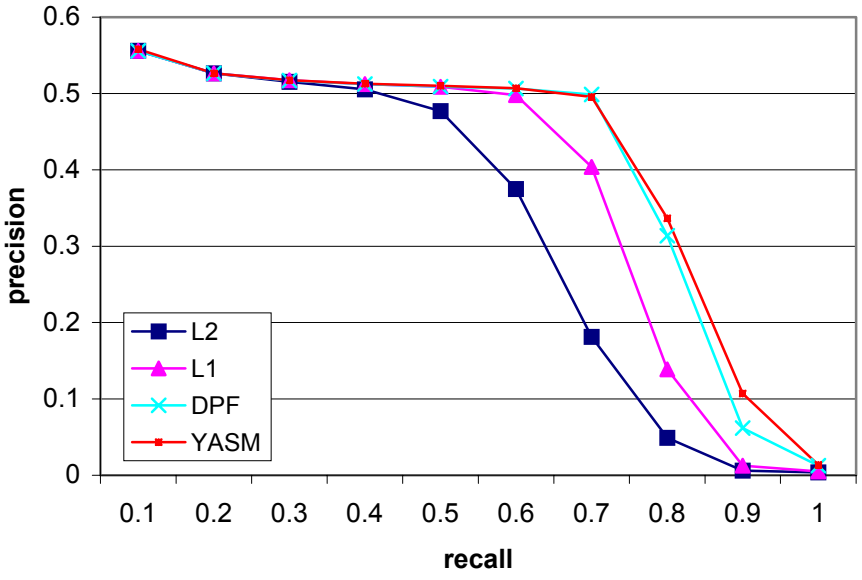



**Figure 1 using seeding images as queries**

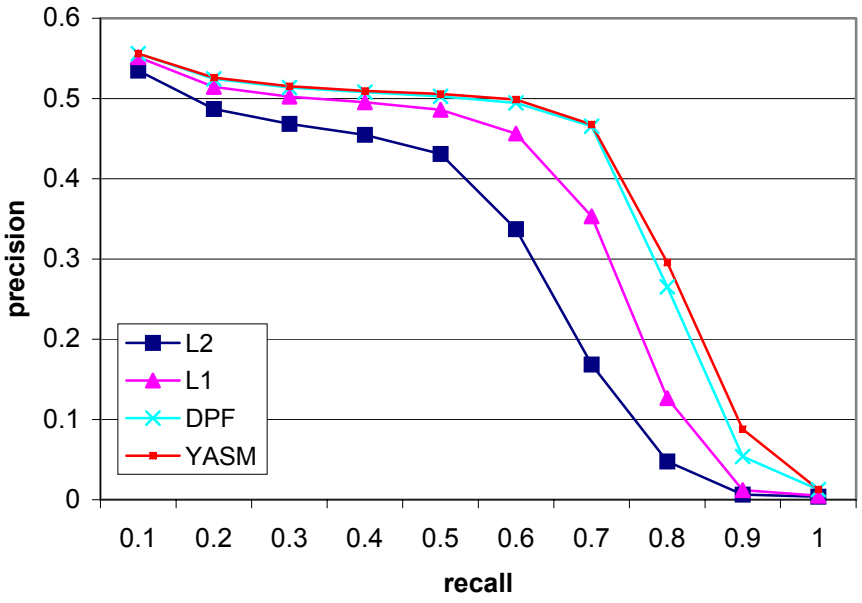



**Figure 2 using GIF images as queries**

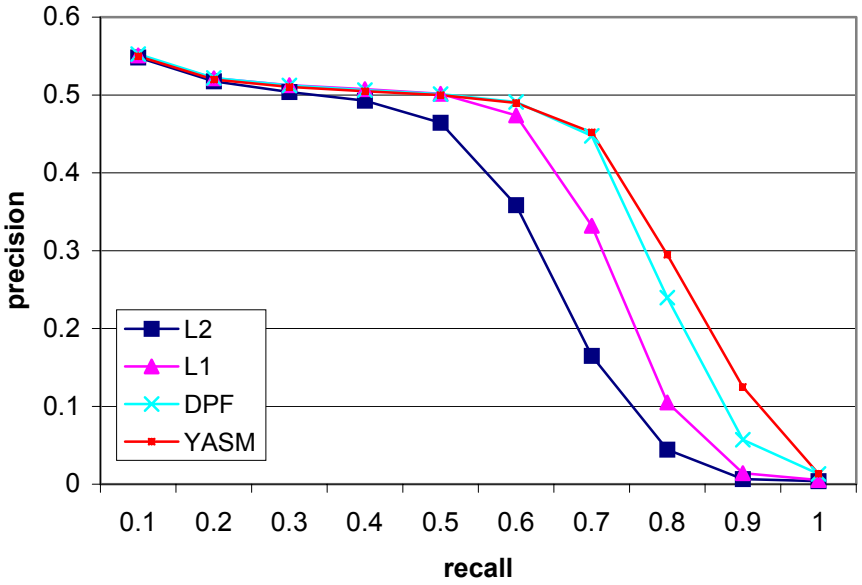



**Figure 3 using crop_5% images as queries**

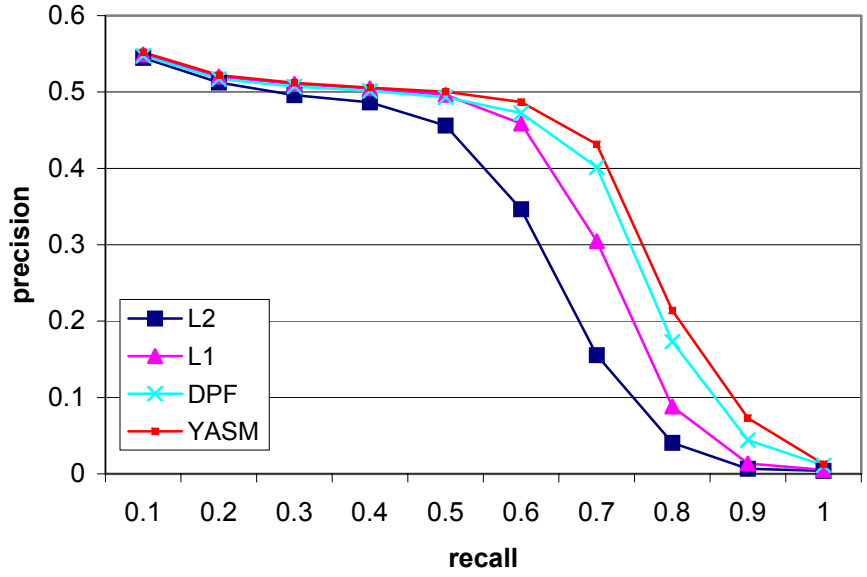



**Figure 4 using crop_10% images as queries**

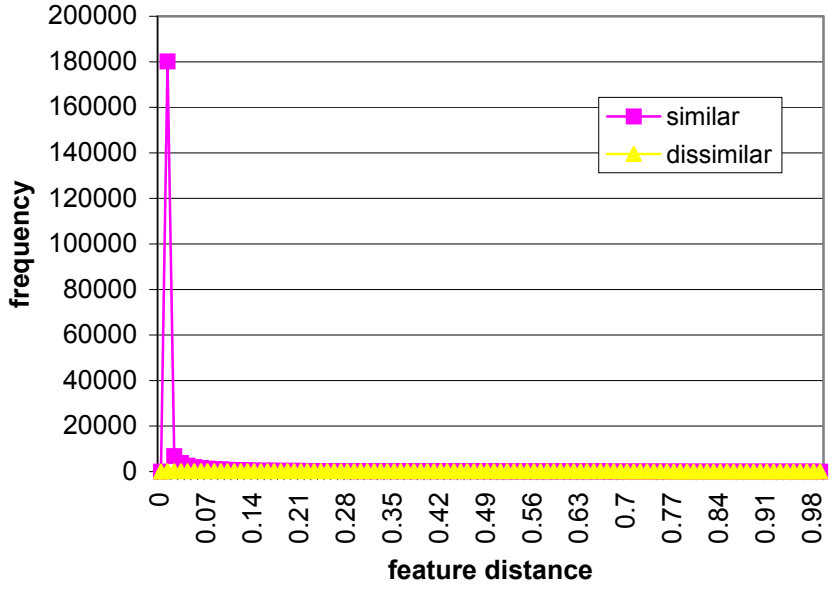



**Figure 5 Feature distance distributions of similar and dissimilar image**

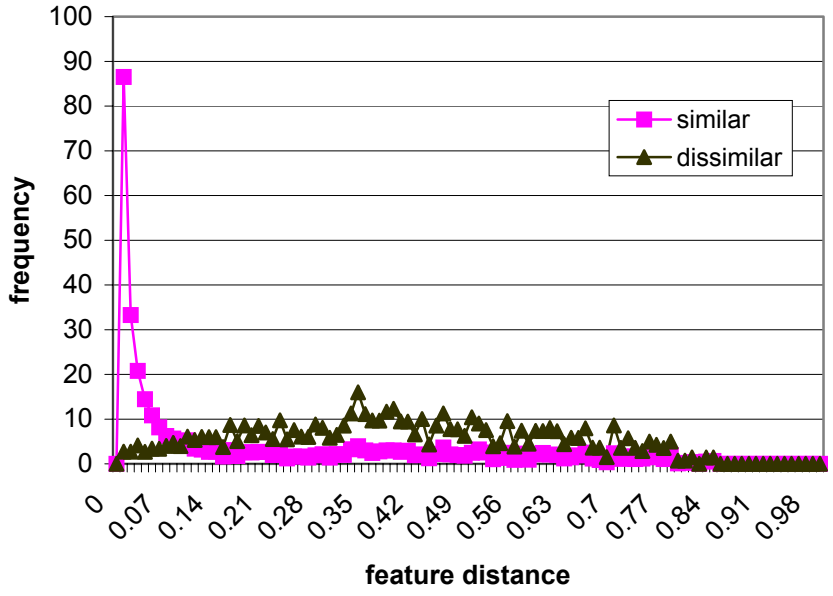



**Figure 6 Feature distance distributions of similar and dissimilar images in the 101st channel**

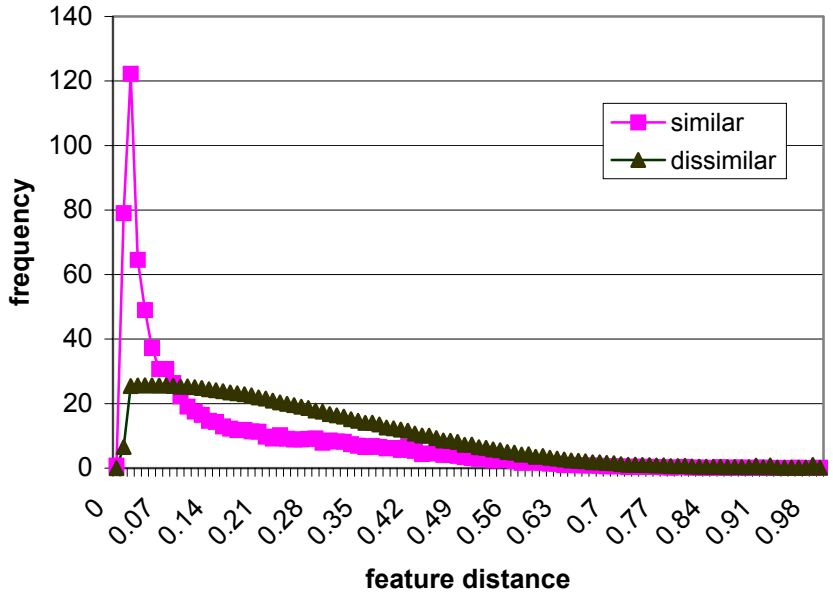



**Figure 7 Feature distance distributions of similar and dissimilar images in the 109th channel**

### 4.2 JND and JNS

We also studied the effect of JND and JNS as Fig 8 and Fig 9 shows. When JND value is small (less than 0.02), the precision is almost the same under the same recall rate. But

when JND increases, the precision becomes higher. When JND is 0.07 and 0.1, the precision could achieve 86% and 97% respectively, which will be unreliable.

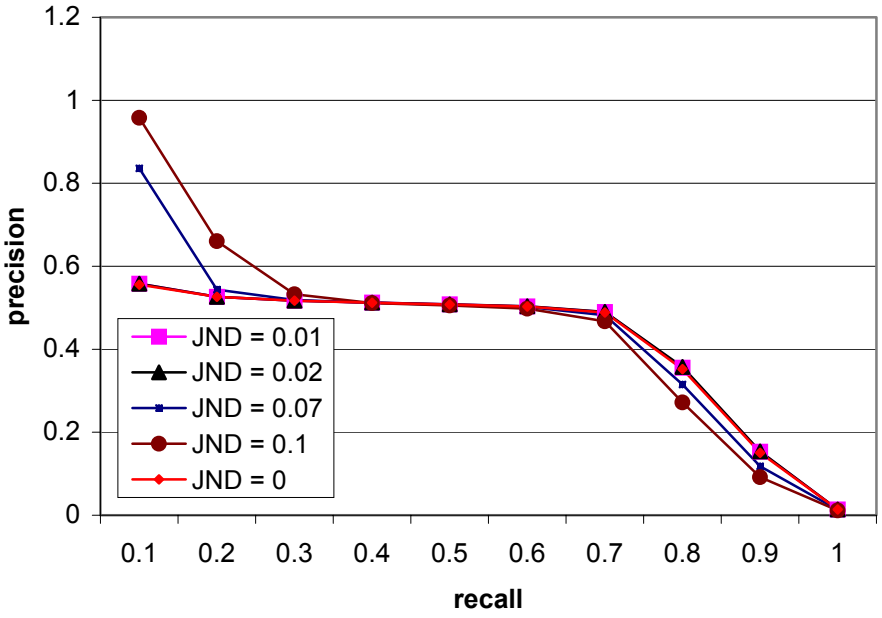



**Figure 8 Changing JND**

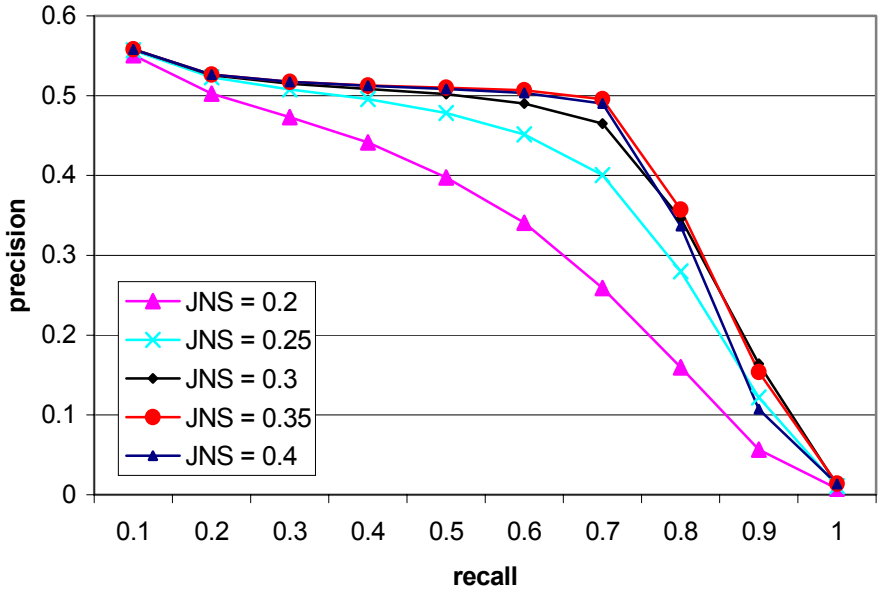



**Figure 9 Changing JNS**

When changing JNS form 0.2 to 0.4, we can see the precision is increasing at the beginning, after it attains the highest (JNS = 0.35), the precision begins to reduce (JNS=0.4). This also confirms that our automatically adjusting JNS method can work

very well. Fig 10 shows the result of our automatically adjusting JNS method. The system could easily find the best JNS value 0.35.

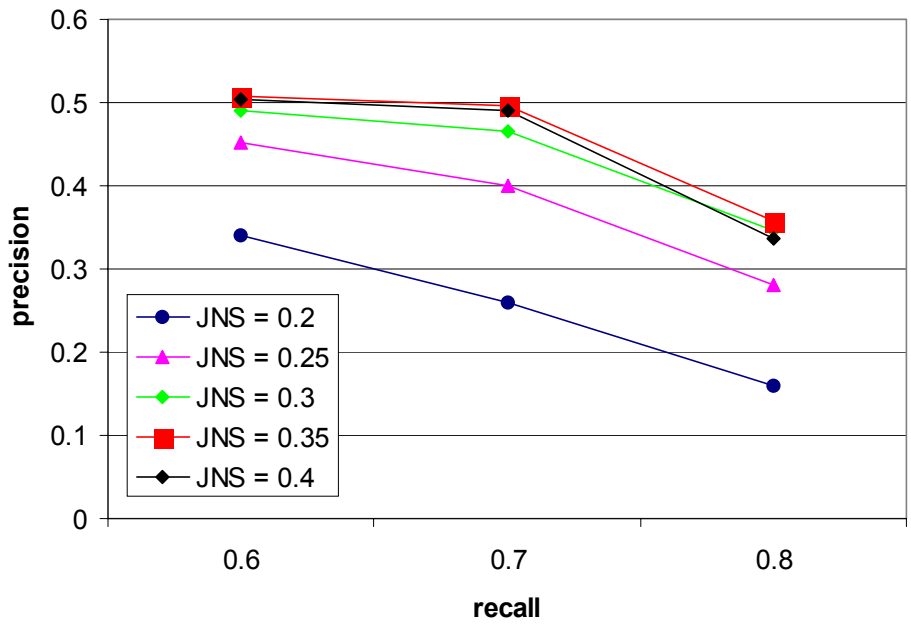



**Figure 10 Automatically adjusting JNS**

### 4.3 Weighted YASM

Considering a user's personality in similarity measure, we also used weighted YASM, in which a weighting vector is learned for each query from the user. Like MARS, we use the inverse of the standard deviation among similar images in each channel as the weight for this channel. Here, we allowed the weighted function to work under the ideal condition – that is, it knows all similar images a prior and can compute the ideal weighting vector for each query. Fig 11 shows that WYASM works even better than YASM. We also compared the WYASM with traditional distance functions using seeding and crop_10% images as queries (Fig 12, and Fig13). All of them use the same weighting vector. WYASM still outperforms the others. Here it should be mentioned that relevance feedback technique can be further employed to capture user's personality, and to improve the system performance.

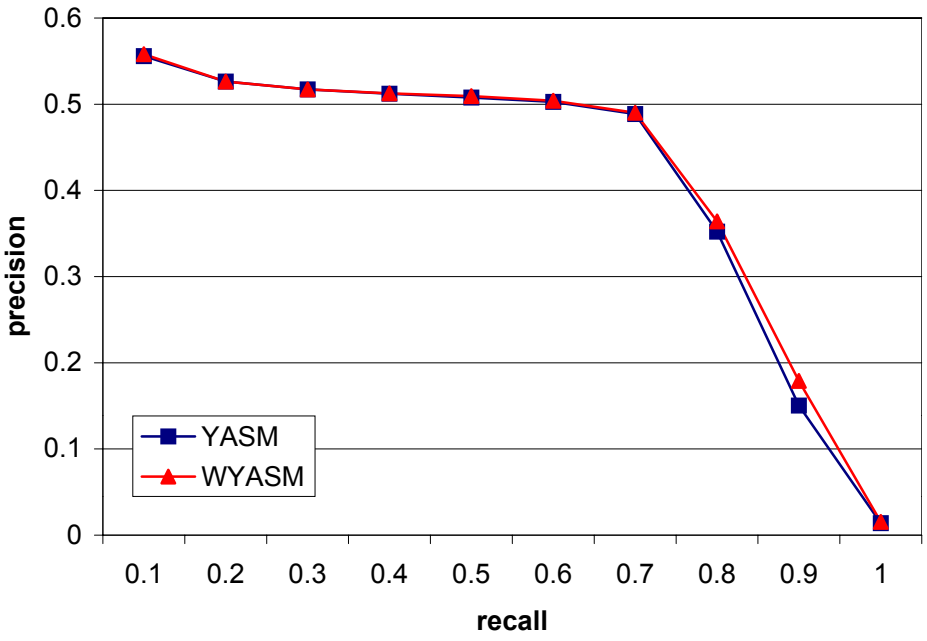



**Figure 11 Comparison between WYASM and YASM**

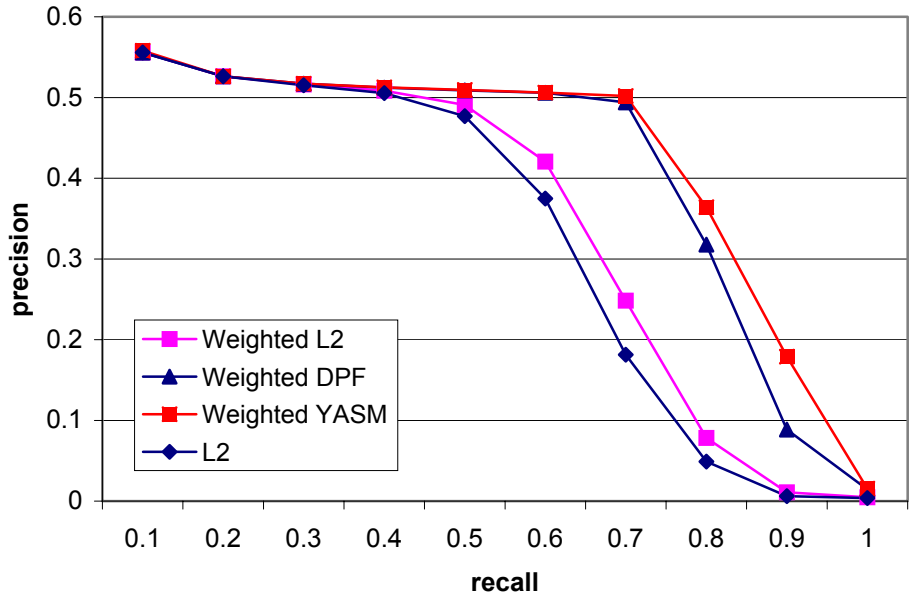



**Figure 12 Comparison of the weighted functions using seeding images as queries**

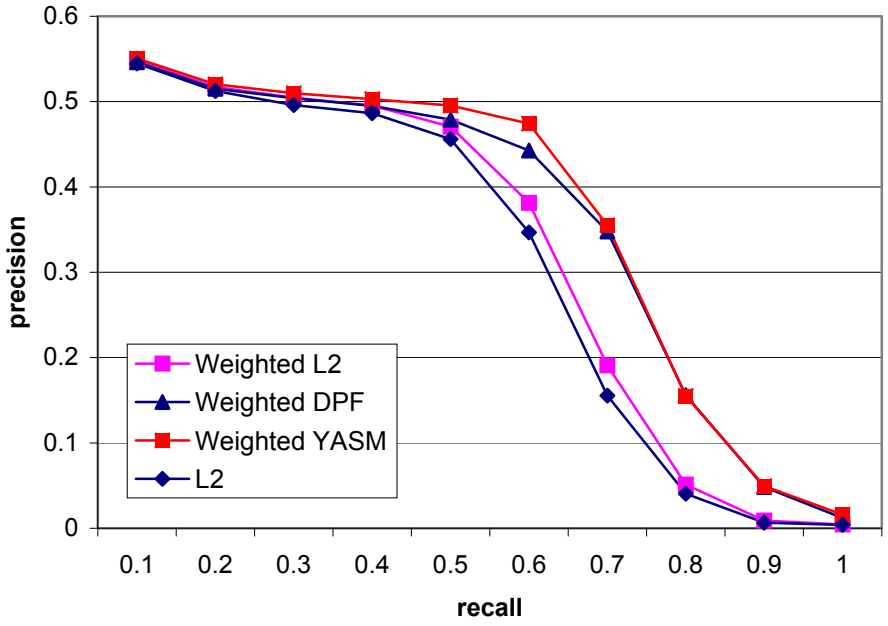



**Figure 13 Comparison of the weighted functions using crop_10% images queries**

### 4.4 Fractional YASM functions

We also studied fractional distance functions [15, 16] in YASM, i.e. the exponent r in distance function is fractional. From Fig 5, Fig 6 and Fig 7, we know that the feature distances of similar images are much skewed toward small values. Using fractional functions, we can further emphasize the importance of the small values (feature distances of the similar channels). But too small exponent leads to the overemphasizing of the small values, and the system performance, in turn, will become worse. In Fig 14, we can see when changing r from 2 to 0.01, the system performance is improved at the beginning, and later when r=0.01, the system performance began to worsen, (the precision-recall curve when r = 0.01 is lower than that of r = 0.1).

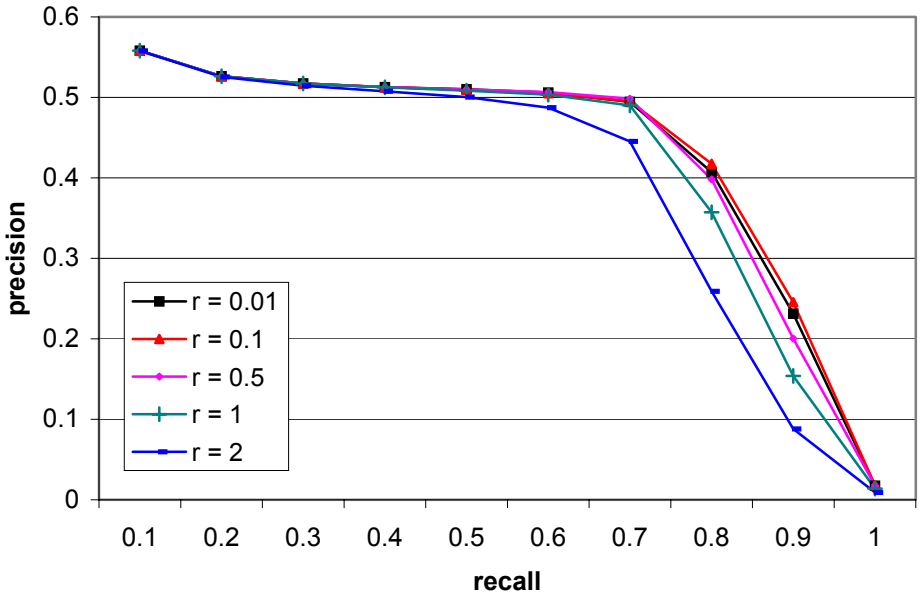



**Figure 14 Fractional YASM functions**

### 4.5 Robustness of YASM

To further test the robustness of YASM, we compared YASM with the traditional distance functions on another dataset, in which we expand the feature vectors to 240 and introduce more noise data (we visually identified them and found some of them are similar, other are not). This dataset consists of 1725 seeding images and 57 kinds of transformed images.

The 57 transformations include:

1. Colorizing.
   We colorize each image through Red/Green/Blue channels by five percentages: 0/0/30%, 0/30%/0, 30%/0/0, 30/70%/90% and 50%/30%/0.
2. Modulation.
   For each image, we vary the amplitude of the hue/saturation/brightness channels by fifteen percentages: 40%/0/0, 60%/0/0, 80%/0/0, 120%/0/0, 140%/0/0, 0/40%/0, 0/60%/0, 0/80%/0, 0/120%/0, 0/140%/0, 0/0/40%, 0/0/60%, 0/0/80%, 0/0/120% and 0/0/140%.
3. Cropping.
   We symmetrically remove the outer borders to reduce each image by 5%, 10%, 20%, 30%, 40%, 50%, 60% and 70%, and then scale it back up to the original size.
4. Rotation.
   We rotate each image by 90, 180 and 270 degrees.
5. Scaling.
   a. Scale up then down. We scale each image up by 5 and 10 times, and then scale it back to the original size.

b. Scale down then up. We scale each image down by factors of 2, 5 and 10, then scale it back to the original size.

6. Downsampling.
   We downsample each image by seven different percentages: 10%, 20%, 30%, 40%, 50%, 70% and 90%.
7. Shading.
   We shade the image using a distant light source, the position of which is specified by the two parameters, azimuth =1000, elevation = 50.
8. Blurring.
   We blur each image using five Gaussian operators, of which the radius/standard deviations based on pixels are: 6/2, 6/3, 6/4, 12/4 and 12/8.
9. Framing.
   For each image, we produce five framed images, each with a randomly selected frame color.
10. Changing format.
    We obtain the GIF version of each JPEG image.
11. Changing to monochrome.
    We transfer each colored image to black and white.
12. Changing contrast.
    We enhance the intensity differences between the lighter and darker elements of each image.

Each image is represented by a 240 dimensional feature vector composed of 144 old and 96 new texture feature components. The new texture features are extracted using a bank of real, circularly symmetric Gabor filters based on its computational efficiency [17, 18]. We use the filter bank with four scales, and the orientation of the filters changes by the step of $\pi/6$. The Gabor kernel sizes are 5x5, 9x9, 17x17 and 35x35. In each scale and each orientation, we used the same texture features: energy mean, energy variance, spreadness, and elongation.

We compared YASM with DPF and Minkowski-type functions using Seeding, Crop_10%, Yellow_frame, Rotation_180degrees, Scaleup_5fold images as queries respectively (Fig 15, Fig 16, Fig 17, Fig 18, and Fig 19). The experimental results show that YASM works the best in all the cases.

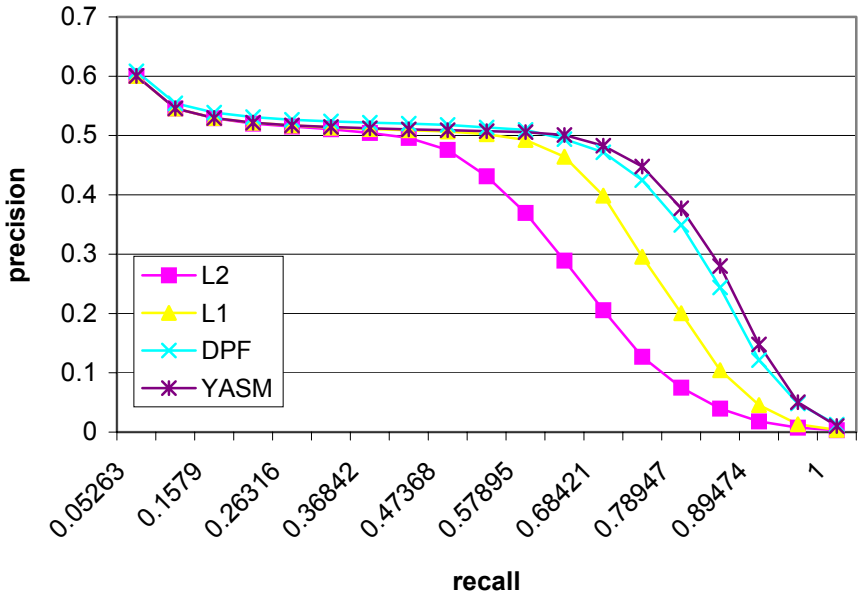



**Figure 15 using seeding images as queries**

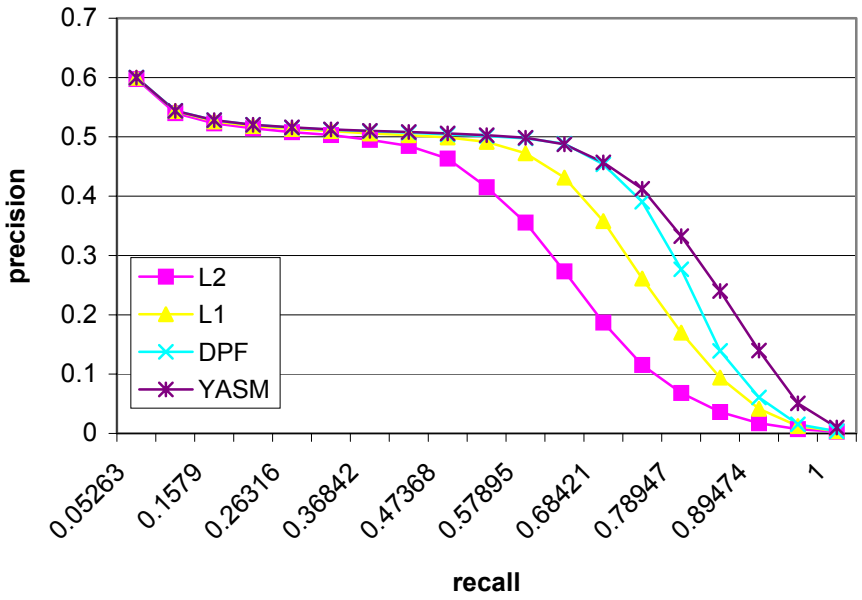



**Figure 16 using crop_10% images as queries**

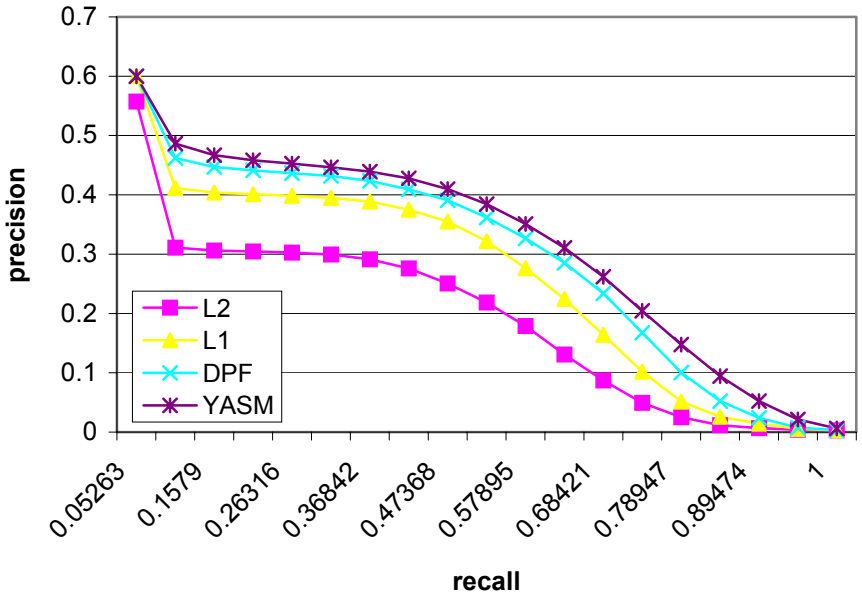



**Figure 17 using Yellow_frame image as queries**

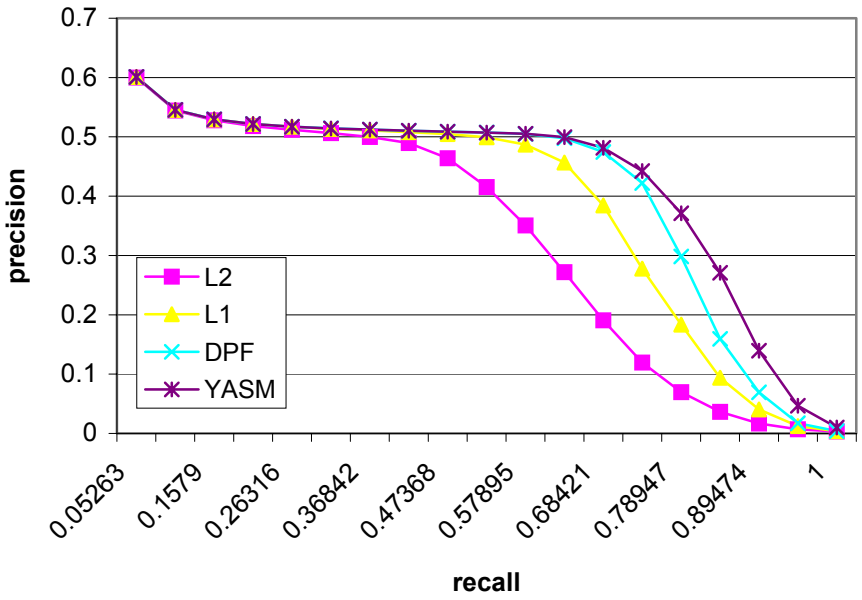



**Figure 18 using rotation_180degrees images as queries**

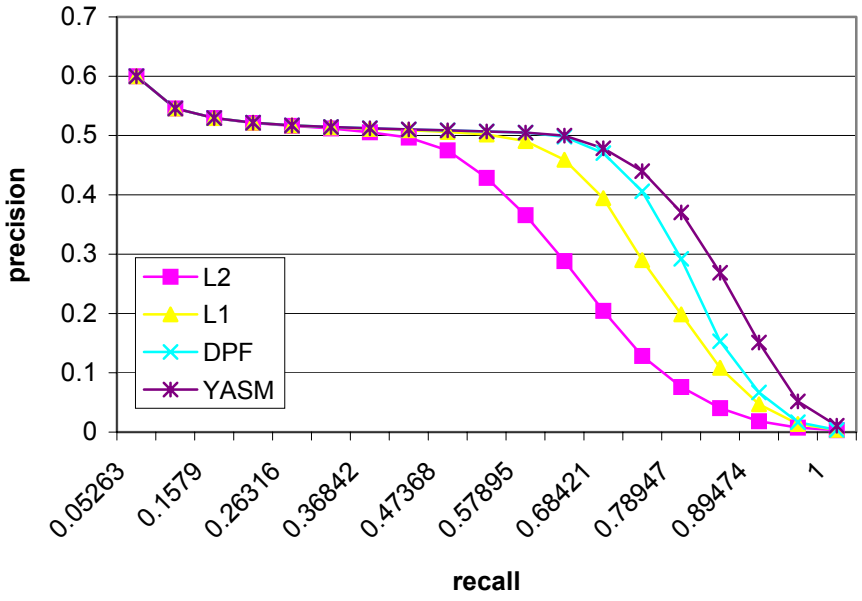



**Figure 19 using scaleup_5fold images as queries**

### 4.6 Further analysis

Also, to further study why DPF does not work better than YASM, we use seeding images as queries and analyze the missing rate of each kind of transformed images. For DPF, the m value is fixed. But Fig 20 shows that to achieve the lowest missing rate, the best m value of the colorized images (m = 120) is different from that of the cropped images (m=220). This indicates that for different object pairs, m values should be different, which is consistent with YASM.

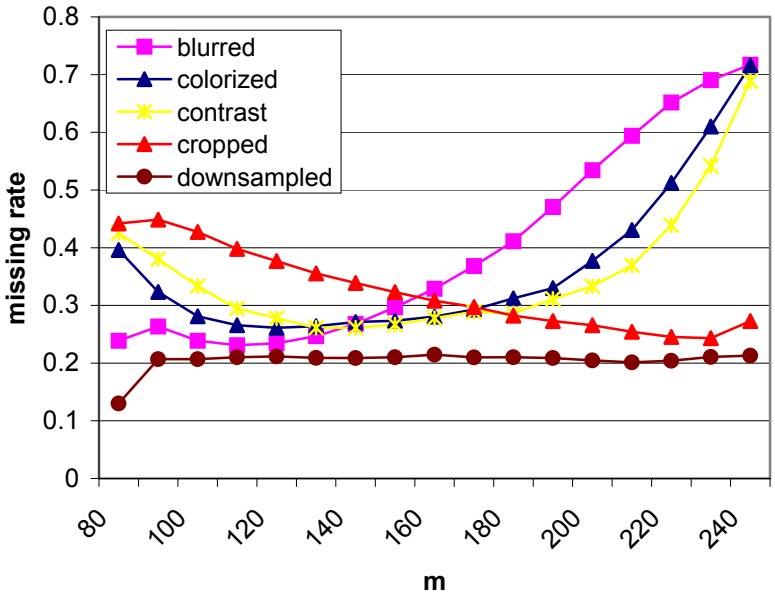



**Figure 20 Mining for the optimal m**

Another advantage is that compared with DPF, YASM does not need to sort all the feature distances when comparing object pairs. So YASM works faster.

## Conclusion and future directions

Similarity is one of the most important theoretical constructs in information retrieval. In this paper, we propose a new method YASM based on the knowledge of psychology theory. It uses the concepts of JND and JNS to select similar features in feature space, which simulate human's perception much better. Compared with traditional similarity measure methods, YASM works much better, both in effectiveness and efficiency.

We plan to extend our study in at least two directions. First, in order to capture user's personality in image retrieval, additional weighting factors should be considered. Second, Since YASM is a non-metric function, non-metric categorizing and indexing schemes need to be investigated.

## References


[1] M.W. Richardson, Multidimensional psychophysics, Psychological Bulletin, 35:659-660, 1938

[2] Y.Rubner, C.Tomasi, and L.J.Guibas, A metric for distributions with application to image databases, Proceedings of 1998 IEEE Conference on Computer Vision, Bombay, India

[3] J.Li, J.Z.Wang, and G. Wiederhold, IRM: Integrated region matching for image retrieval. Proceedings of ACM Multimedia, October, 2000.

[4] B. Li, E. Chang, The Discovery of A Percpeptual Distance Function for Measuring Image Similarity, ACM Multimedia Journal Special Issue (accepted), 2002.

[5] A. Tversky, Features of similarity, Psychological Review, 84(4):327-352, 1977

[6] E.B.Goldstein, Sensation & perception, (fifth edition), Brooks/Cole, pp8-11, 1999

[7] E. Chang, B. Li and C. Li, Toward perception based image retrieval,

[8] Y.Chen, J.Z.Wang, A region-based fuzzy matching approach to content-based image retrieval, IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 24, no. 9, pp. -, 2002

[9] F.P.Hughes, and L.D.Noppe, "Human development across the life span", Macmillan, 1990

[10] Y. Rui, T.S. Huang and S. Mehrotra, Content based image retrieval with relevance feedback in mars, ICIP (2) 1997: 815-818

[11] S. Tong and E. Chang, Support vector machine active learning for image retrieval. Proceedings of ACM International Conference on Multimedia, pp.107-118, October, 2001

[12] J.Z. Wang, G. Wiederhold, O. Firschein, and S. X. Wei, content based image indexing and searching using daubechies' wavelets. Journal of Digital Libraries, 1(4):311-328, 1998

[13] J.G.Leu, Computing a shape's moments from its boundary. Pattern Recognition, 24(10), pp949-957, 1991

[14] J.Smith and S.F. Chang, Automated image retrieval using color and texture, IEEE Transactions on Pattern Analysis and Machine Intelligence, November, 1996.

[15] A.Hinneburg, C.C. Aggarwal, and D.A. Keim, What is the nearest neighbor in high dimensional spaces?, VLDB 2000: 506-515

[16] C.C. Aggarwal, A.Hinneburg and D.A. Keim, On the surprising behavior of distance metrics in high dimensional space, ICDT 2001: 420-434

[17] A. Jain and G. Healey, "A multiscale Representation Including Opponent Color features for texture recognition", IEEE trans. on Image Processing, 7(1):124-128, 1998

[18] B.S. Manjunath and P. Wu, S. Newsam, and H. Shin, A texture descriptor for browsing and similarity retrieval, Signal Processing Image Communications, 2001